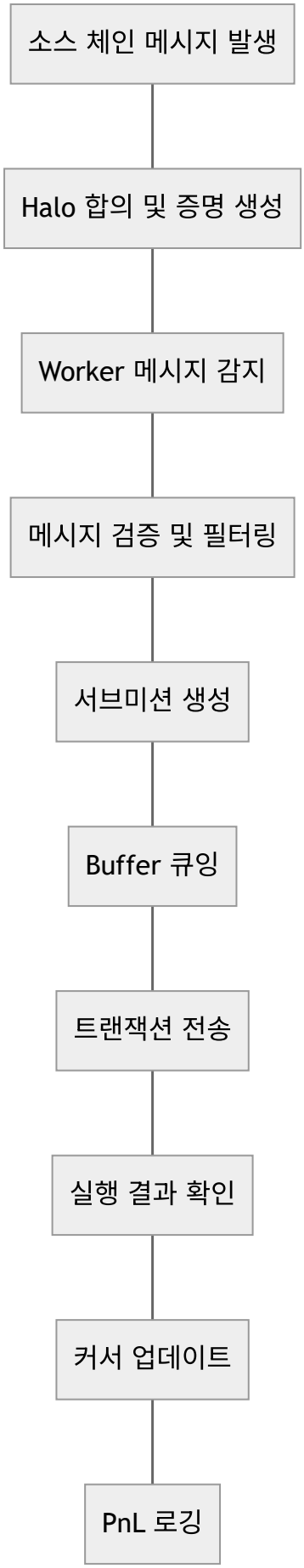
09. Relayer 크로스체인 메시지 처리 플로우 분석

개요

이 문서는 Relayer가 크로스체인 메시지를 감지, 검증, 전송하는 전체 플로우를 상세히 분석합니다. Relayer는 소스 체인에서 발생한 메시지를 목적지 체인으로 안전하게 전달하는 핵심 역할을 수행합니다.

전체 메시지 처리 플로우



1. Worker 실행 플로우

Worker.Run() - 메인 실행 루프

```
func (w *Worker) Run(ctx context.Context) {
ctx = log.WithCtx(ctx, "dst_chain", w.destChain.Name)
backoff := expbackoff.NewWithAutoReset(ctx)
for ctx.Err() == nil {
err := w.runOnce(ctx)
if ctx.Err() != nil {
return
} else if errors.Is(err, chaos.ErrChaos) {
log.InfoErr(ctx, "Worker failed due to chaos testing, resetting", err)
} else {
log.Warn(ctx, "Worker failed, resetting", err)
}

workerResets.WithLabelValues(w.destChain.Name).Inc()
backoff()
}
}
```

특징:

- 무한 루프로 실행
- 실패시 지수 백오프로 재시도

- Chaos Engineering 지원

Worker.runOnce() - 단일 실행 사이클

```
func (w *Worker) runOnce(ctx context.Context) error {
log.Info(ctx, "Worker starting")

// 1. 제출된 커서 조회
cursors, err := getSubmittedCursors(ctx, w.network, w.destChain.ID, w.xProvider)
if err != nil {
return err
}

// 2. Sender 초기화
sender, err := w.sendProvider()
if err != nil {
return err
}

// 3. Active Buffer 생성
buf := newActiveBuffer(w.destChain.Name, mempoolLimit, sender)

// 4. 증명 오프셋 계산
attestOffsets, err := fromChainVersionOffsets(cursors, w.network.ChainVersionsTo
```

```go
if err != nil {
return err
}

// 5. 부트스트랩 오프셋 적용
for chainVer, offset := range attestOffsets {
bootstrapOffset, ok := getBootstrapOffset(w.network, w.destChain, chainVer)
if ok && bootstrapOffset > offset {
attestOffsets[chainVer] = bootstrapOffset
}
}

// 6. 저장된 커서 적용
stored, err := w.cursors.WorkerOffsets(ctx, w.destChain.ID)
if err != nil {
return err
}
for chainVer, offset := range attestOffsets {
if stored[chainVer] > offset {
attestOffsets[chainVer] = stored[chainVer]
}
}
```

```go
// 7. 메시지 필터 초기화
msgFilter, err := newMsgOffsetFilter(cursors)
if err != nil {
return err
}

// 8. 각 체인 버전별 스트리밍 시작
for chainVer, fromOffset := range attestOffsets {
callback := w.newCallback(
msgFilter,
buf.AddInput,
newMsgStreamMapper(w.network),
chainVer,
)

w.cProvider.StreamAsync(ctx, chainVer, fromOffset, w.destChain.Name, callback)
}

// 9. Buffer 실행
return buf.Run(ctx)
}
```

2. 메시지 감지 및 콜백 처리

newCallback() - 증명 처리 콜백

```go
func (w *Worker) newCallback(
msgFilter *msgCursorFilter,
sendBuffer  func(context.Context, xchain.Submission) error,
msgStreamMapper msgStreamMapper,
streamerChainVer xchain.ChainVersion,
) cchain.ProviderCallback {
var  cachedValSetID  uint64
var  cachedValSet []cchain.PortalValidator

return  func(ctx context.Context, att xchain.Attestation) error {
// 1. XBlock 조회
block, ok, err := fetchXBlock(ctx, w.xProvider, att)
if  err != nil {
return  err
} else  if !ok {
return  nil  // 퍼지 증명 불일치, 스킵
} else  if  len(block.Msgs) == 0 {
// 메시지 없음, 커서만 업데이트
return  saveCursors(nil)
```

```
}

// 2. 머클 트리 생성
msgTree, err := xchain.NewMsgTree(block.Msgs)
if err != nil {
return err
}

// 3. 검증자 세트 캐싱
if att.ValidatorSetID != cachedValSetID {
cachedValSetID = att.ValidatorSetID
cachedValSet, ok, err = w.cProvider.PortalValidatorSet(ctx, att.ValidatorSetID)
if err != nil {
return errors.Wrap(err, "fetch validator set")
} else if !ok {
return errors.New("validator set not found [BUG]", "valset", att.ValidatorSetID
}
}

// 4. 스트림별 메시지 처리
applicable := make(map[xchain.StreamID][]xchain.Msg)
for streamID, msgs := range msgStreamMapper(block.Msgs) {
if streamID.DestChainID != w.destChain.ID {
```

```
continue  // 다른 목적지 체인 스킵
} else  if !attestationForShard(att, streamID.ShardID) {
continue  // 샤드 불일치 스킵
}

applicable[streamID] = msgs

// 5. 검증자 세트 대기
if  err := w.awaitValSet(ctx, att.ValidatorSetID);  err != nil {
return  errors.Wrap(err, "await validator set")
}

// 6. 메시지 필터링
msgs, err = filterMsgs(ctx, streamID, w.network.StreamName, msgs, msgFilter)
if  err != nil {
return  err
} else  if  len(msgs) == 0 {
continue
}

// 7. 서브미션 생성
update := StreamUpdate{
StreamID: streamID,
```

```
Attestation: att,
Msgs: msgs,
MsgTree: msgTree,
ValSet: cachedValSet,
}

submissions, err := w.creator(update)
if err != nil {
return err
}

// 8. Buffer에 전송
for _, subs := range submissions {
if err := sendBuffer(ctx, subs); err != nil {
return err
}
}
}

return saveCursors(applicable)
}
}
```

3. 메시지 스트림 매핑

newMsgStreamMapper() - 메시지 스트림 분류

```
func newMsgStreamMapper(network netconf.Network) msgStreamMapper {
consensusChain, _ := network.OmniConsensusChain()

return func(msgs []xchain.Msg) map[xchain.StreamID][]xchain.Msg {
resp := make(map[xchain.StreamID][]xchain.Msg)
for _, msg := range msgs {
// 일반 메시지는 스트림 ID로 매핑
if msg.SourceChainID != consensusChain.ID {
resp[msg.StreamID] = append(resp[msg.StreamID], msg)
continue
}

// 합의 체인 메시지는 모든 EVM 체인으로 브로드캐스트
for _, evmChain := range network.EVMChains() {
streamID := xchain.StreamID{
SourceChainID: consensusChain.ID,
DestChainID: evmChain.ID,
ShardID: msg.ShardID,
}
```

```
resp[streamID] = append(resp[streamID], msg)
}
}

return resp
}
}
```

메시지 분류 규칙:

- **일반 메시지**: 특정 스트림 ID로 매핑
- **합의 체인 메시지**: 모든 EVM 체인으로 브로드캐스트

4. 메시지 필터링

filterMsgs() - 중복 메시지 필터링

```
func filterMsgs(
ctx context.Context,
streamID xchain.StreamID,
streamNamer func(xchain.StreamID) string,
msgs []xchain.Msg,
msgFilter *msgCursorFilter,
) ([]xchain.Msg, error) {
res := make([]xchain.Msg, 0, len(msgs))
```

```go
for i := 0; i < len(msgs); {
msg := msgs[i]

check, cursor := msgFilter.Check(streamID, msg.StreamOffset)
if check == checkProcess {
res = append(res, msg)
} else if check == checkIgnoreOffset && !msg.ShardID.ConfLevel().IsFuzzy() {
log.Debug(ctx, "Filtering finalized msg already delivered",
"stream", streamNamer(streamID),
"offset", msg.StreamOffset,
"cursor_offset", cursor.MsgOffset,
)
}
if check != checkGapOffset {
i++
continue
}

// 퍼지 확인 레벨의 갭 처리
if !streamID.ConfLevel().IsFuzzy() {
return nil, errors.New("unexpected gap in finalized msg offsets [BUG]")
}
```

```
log.Warn(ctx, "Gap in fuzzy msg offsets, waiting for ConfFinalized", nil,
"stream", streamNamer(streamID),
"offset", msg.StreamOffset,
"cursor_offset", cursor.MsgOffset,
)
backoff()
// 같은 메시지 재시도
}

return res, nil
}
```

필터링 결과:

- `checkProcess` : 메시지 처리
- `checkIgnoreOffset` : 이미 처리된 메시지 무시
- `checkGapOffset` : 갭 발생시 대기 (퍼지 확인 레벨만)

5. 서브미션 생성

CreateSubmissions() - 서브미션 분할 생성

```
func CreateSubmissions(up StreamUpdate) ([]xchain.Submission, error) {
// 1. 입력 검증
for i, msg := range up.Msgs {
```

```go
if msg.SourceChainID != up.SourceChainID {
return nil, errors.New("invalid msgs [BUG]")
} else if i > 0 && msg.StreamOffset != up.Msgs[i-1].StreamOffset+1 {
return nil, errors.New("msgs not sequential [BUG]")
}
}

// 2. 쿼럼 서명 선택
sigs, err := quorumSigs(up.ValSet, up.Attestation.Signatures)
if err != nil {
return nil, errors.Wrap(err, "quorum sigs")
}

// 3. 증명 루트 계산
attRoot, err := xchain.AttestationRoot(att.AttestHeader, att.BlockHeader, att.Ms
if err != nil {
return nil, err
}

// 4. 메시지 그룹화 및 서브미션 생성
var resp []xchain.Submission
for _, msgs := range groupMsgsByCost(up.Msgs) {
multi, err := up.MsgTree.Proof(msgs)
```

```
if err != nil {
return nil, err
}

resp = append(resp, xchain.Submission{
AttestationRoot: attRoot,
ValidatorSetID: att.ValidatorSetID,
AttHeader: att.AttestHeader,
BlockHeader: att.BlockHeader,
Msgs: msgs,
Proof: multi.Proof,
ProofFlags: multi.ProofFlags,
Signatures: sigs,
DestChainID: up.DestChainID,
})
}

return resp, nil
}
```

quorumSigs() - 최소 쿼럼 서명 선택

```
func quorumSigs(valSet []cchain.PortalValidator, signatures []xchain.SigTuple)
```

```go
var totalPower int64
powers := make(map[common.Address]int64)
for _, val := range valSet {
totalPower += val.Power
powers[val.Address] = val.Power
}

quorum := totalPower * 2 / 3

// 파워 내림차순 정렬
sort.Slice(signatures, func(i, j int) bool {
return powers[signatures[i].ValidatorAddress] > powers[signatures[j].ValidatorA
})

// 쿼럼 달성까지 최소 서명 선택
var resp []xchain.SigTuple
var sum int64
for _, sig := range signatures {
resp = append(resp, sig)
power, ok := powers[sig.ValidatorAddress]
if !ok {
return nil, errors.New("signature not in validator set [BUG]")
}
```

```
sum += power
if sum > quorum {
break
}
}

return resp, nil
}
```

groupMsgsByCost() - 가스 비용 기반 메시지 그룹화

```
func groupMsgsByCost(msgs []xchain.Msg) [][]xchain.Msg {
var resp [][]xchain.Msg
var current []xchain.Msg
for _, msg := range msgs {
// 메시지 추가시 최대 가스 한도 초과 여부 확인
if naiveSubmissionGas(append(slices.Clone(current), msg)) > subGasMax {
resp = append(resp, current)
current = nil
}
current = append(current, msg)
}
```

```
return append(resp, current)
}
```

6. 비동기 버퍼 처리

activeBuffer.Run() - 버퍼 실행

```
func (b *activeBuffer) Run(ctx context.Context) error {
sema := semaphore.NewWeighted(b.mempoolLimit)
for {
select {
case <-ctx.Done():
return errors.Wrap(ctx.Err(), "context canceled")
case err := <-b.errChan:
return err
case submission := <-b.buffer:
// 1. 세마포어 획득 (멤풀 제한)
if err := sema.Acquire(ctx, 1); err != nil {
return errors.Wrap(err, "acquire semaphore")
}
mempoolLen.WithLabelValues(b.chainName).Inc()

// 2. 비동기 전송 시작 (논스 순서 보장)
```

```
response := b.sendAsync(ctx, submission)
go func() {
err := <-response
if err != nil {
b.submitErr(errors.Wrap(err, "send submission"))
}


mempoolLen.WithLabelValues(b.chainName).Dec()
sema.Release(1)
}()


// 3. Chaos 테스트
if err := chaos.MaybeError(ctx); err != nil {
return err
}
}
}
}
```

버퍼 특징:

- **멤풀 제한**: 동시 트랜잭션 수 제한
- **논스 순서**: 동기적 논스 예약으로 순서 보장
- **비동기 처리**: 트랜잭션 전송과 확인 분리

7. 트랜잭션 전송

Sender.SendAsync() - 비동기 트랜잭션 전송

```
func (s Sender) SendAsync(ctx context.Context, sub xchain.Submission) <-chan  er

// 1. 논스 예약 (동기적)

nonce, err := s.txMgr.ReserveNextNonce(ctx)

if  err != nil {

return  returnErr(err)

}



// 2. 가스 추정

estimatedGas := s.gasEstimator(s.chain.ID, sub.Msgs)



// 3. 트랜잭션 데이터 인코딩

txData, err := xchain.EncodeXSubmit(xchain.SubmissionToBinding(sub))

if  err != nil {

return  returnErr(err)

}



// 4. 트랜잭션 후보 생성

candidate := txmgr.TxCandidate{

TxData: txData,
```

```
To: &s.chain.PortalAddress,
GasLimit: estimatedGas,
Value: bi.Zero(),
Nonce: &nonce,
}

// 5. 비동기 전송
asyncResp := make(chan  error, 1)
go  func() {
tx, rec, err := s.txMgr.Send(ctx, candidate)
if  err != nil {
asyncResp <- errors.Wrap(err, "failed to send tx")
return
}

// 6. 메트릭 업데이트
submissionTotal.WithLabelValues(srcChain, dstChain).Inc()
msgTotal.WithLabelValues(srcChain, dstChain).Add(float64(len(sub.Msgs)))
gasEstimated.WithLabelValues(dstChain).Observe(float64(estimatedGas))

// 7. PnL 로깅
if  s.onSubmit != nil {
go  s.onSubmit(ctx, tx, rec, sub)
```

```
}

// 8. 실행 결과 확인
if  rec.Status == ethtypes.ReceiptStatusFailed {
revertedSubmissionTotal.WithLabelValues(srcChain, dstChain).Inc()
asyncResp <- errors.New("submission reverted")
return
}

asyncResp <- nil
}()

return  asyncResp
}
```

8. 가스 추정

newGasEstimator() - 가스 추정 함수

```
func  newGasEstimator(network netconf.ID) gasEstimator {
consChainID := network.Static().OmniConsensusChainIDUint64()

// Arbitrum은 비표준 가스 사용으로 모델 스킵
```

```go
skipModel := map[uint64]bool{
evmchain.IDArbSepolia: true,
evmchain.IDArbitrumOne: true,
}

return func(destChain uint64, msgs []xchain.Msg) uint64 {
if len(msgs) == 0 {
return subGasBase
}

srcChain := msgs[0].SourceChainID

if skipModel[destChain] {
return properGasEstimation // RPC 가스 추정 사용
}

if srcChain == consChainID {
if network.IsEphemeral() {
return subEphemeralConsensusGas // 임시 체인용 고정값
}
return properGasEstimation // 보호된 체인용 RPC 추정
}
```

```
return naiveSubmissionGas(msgs) // 단순 모델 사용
}
}
```

naiveSubmissionGas() - 단순 가스 모델

```
func naiveSubmissionGas(msgs []xchain.Msg) uint64 {
resp := subGasBase
for _, msg := range msgs {
resp += msg.DestGasLimit + subGasXmsgOverhead
}
return resp
}
```

가스 추정 전략:

- **Arbitrum**: RPC 추정 (비표준 가스 모델)
- **합의 체인 메시지**: 네트워크 타입별 다른 전략
- **일반 메시지**: 단순 모델 (기본 + 메시지별 가스)

9. 커서 관리

Store.Insert() - 커서 저장

```
func (s *Store) Insert(
```

```go
ctx context.Context,
srcVersion xchain.ChainVersion,
destChain  uint64,
attestOffset  uint64,
submittedMsgs  map[xchain.StreamID][]xchain.Msg,
) error {
// 1. 샤드별 최고 오프셋 계산
offsetsByShard := make(map[uint64]uint64)
for  streamID, msgs := range  submittedMsgs {
if  len(msgs) == 0 {
continue
}
offsetsByShard[uint64(streamID.ShardID)] = msgs[len(msgs)-1].StreamOffset
}

// 2. 커서 생성
c := &Cursor{
SrcChainId: srcVersion.ID,
ConfLevel: uint32(srcVersion.ConfLevel),
DstChainId: destChain,
AttestOffset: attestOffset,
Confirmed: false,
StreamOffsetsByShard: offsetsByShard,
```

```
}

// 3. 데이터베이스 저장
err := s.db.Insert(ctx, c)
if errors.Is(err, ormerrors.AlreadyExists) {
// 기존 커서와 오프셋 일치 확인
existing, err := s.db.Get(ctx, srcVersion.ID, uint32(srcVersion.ConfLevel), dest
if err != nil {
return errors.Wrap(err, "get cursor")
} else if !maps.Equal(offsetsByShard, existing.GetStreamOffsetsByShard()) {
log.Error(ctx, "Unexpected existing cursor offset mismatch", nil)
}
return nil
}

return err
}
```

confirmOnce() - 커서 확인

```
func (s *Store) confirmOnce(ctx context.Context) error {
all, err := listAll(ctx, s.db)
if err != nil {
```

```go
return errors.Wrap(err, "listAll all cursors")
}

for streamer, cursors := range splitByStreamer(all) {
for _, c := range cursors {
if c.GetConfirmed() {
continue
}

var unconfirmed bool
for shardID, offset := range c.GetStreamOffsetsByShard() {
stream := xchain.StreamID{
SourceChainID: c.GetSrcChainId(),
DestChainID: c.GetDstChainId(),
ShardID: xchain.ShardID(shardID),
}
// 최종화된 제출 커서 확인
submitted, ok, err := s.submitCursorFunc(ctx, xchain.FinalizedRef, stream)
if err != nil || !ok || submitted.MsgOffset < offset {
unconfirmed = true
break
}
}
```

```go
if unconfirmed {
break
}

// 커서 확인 처리
c.Confirmed = true
if err := s.db.Save(ctx, c); err != nil {
return errors.Wrap(err, "save cursor")
}
}
}

return nil
}
```

10. PnL (손익) 로깅

pnlLogger.log() - 손익 기록

```go
func (l pnlLogger) log(ctx context.Context, tx *ethtypes.Transaction, receipt *e
// 1. 지출 계산
spendGwei := totalSpendGwei(tx, receipt)
```

```
spend, err := spendByDenom(dest, spendGwei, prices)
if  err != nil {
return  errors.Wrap(err, "get spend")
}

// 2. 지출 로깅
pnl.Log(ctx,
pnl.LogP{
Type: pnl.Expense, AmountGwei: spend.nUSD, Currency: pnl.USD,
Category: "gas", Subcategory: "xsubmit",
Chain: dest.Name, ID: id, Metadata: md,
},
// ETH, NOM 지출도 동일하게 로깅
)

// 3. 수수료 수입 로깅 (합의 체인 제외, 성공한 트랜잭션만)
if !netconf.IsOmniConsensus(l.network, srcChainID) && receipt.Status == ethtypes
fees, err := feeByDenom(src, sub, prices)
if  err != nil {
return  errors.Wrap(err, "get fees")
}

pnl.Log(ctx,
```

```
pnl.LogP{
Type: pnl.Income, AmountGwei: fees.nUSD, Currency: pnl.USD,
Category: "fees", Subcategory: "xcall",
Chain: src.Name, ID: id, Metadata: md,
},
// ETH, NOM 수입도 동일하게 로깅
)
}
}
```

메시지 처리 플로우 요약

1. **메시지 감지**: CChain Provider를 통한 증명 스트리밍
2. **메시지 검증**: XBlock 조회 및 머클 트리 검증
3. **메시지 필터링**: 중복 제거 및 순서 보장
4. **서브미션 생성**: 가스 제한 고려한 메시지 그룹화
5. **비동기 전송**: 멤풀 제한과 논스 순서 보장
6. **실행 확인**: 트랜잭션 성공/실패 처리
7. **커서 관리**: 진행 상황 추적 및 복구 지원
8. **PnL 추적**: 비용과 수익 모니터링

이러한 플로우를 통해 Relayer는 안전하고 효율적인 크로스체인 메시지 전달을 보장합니다.